LABORATÓRIO 3 parte A

Sistemas Distribuídos | 2025-2026 | Licenciatura em Informática | FCT UAC

(*) = OPCIONAL

OBJETIVOS DE APRENDIZAGEM: Aprender a criar um servidor multi-thread. Partindo do exercício anterior, perceber como o novo servidor passa a aceitar mais do que um cliente ligado. Aplicar estas aprendizagens ao exemplo da máquina de calcular **CALCULADORA_5 .** Criar um espaço para dados. Usar os ferrolhos (ou *lock()*) para gerir a informação utilizada por mais do que uma thread na **CALCULADORA_6**.

1. Programa CALCULADORA_5

O programa apresenta a estrutura anterior da calculadora 4. Deverá usar a versão que criou no último laboratório ou a versão (ainda que incompleta) disponibilizada no laboratório. Deverá efetuar as seguintes transformações do código:

1 (a) Criar o ficheiro processa_cliente.py

Este novo ficheiro, no lado do servidor, irá corresponder a uma thread.

i. Introduza o código deste ficheiro que obedece à seguinte estrutura:

```
class ProcessaCliente(threading.Thread):
    def _init_(self, connection, address): super().
        init_()
        self.connection = connection
        self.address = address
        self.sum = Somar()
       ...

      #----------interaction with sockets ---------------
      ...
      # all functions interacting with sockets
      ...
      #--------
    def run(self):
        print(self.address, "Thread iniciada")
        last_request = False
        while not last_request:
            request_type = self.receive_str(servidor.COMMAND_SIZE)
            if request_type == servidor.ADD_OP:
                x = self.receive_int(servidor.INT_SIZE)
                y = self.receive_int(servidor.INT_SIZE)
                print(f"[{self.address}] Somar: {x} + {y}")
                result = self.sum.execute(x, y)
                self.send_int(result, servidor.INT_SIZE)
            elif request_type == servidor.SUB_OP:
               ...

            elif request_type == servidor.END_OP:
                last_request = True
        print(self.address,"Thread terminada")
        self.connection.close()
```

Pode verificar que esta classe herda a classe Thread. E redefine a função *_init_()* e *run()*. Ela é executada quando a maquina.py chama a função start() (como iremos ver de seguida).

Resposta:

processa_cliente.py

servidor > gestor > processa_cliente.py > ProcessaCliente > run

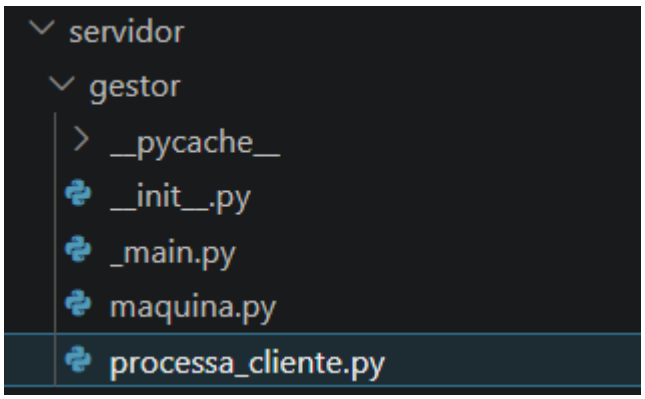
```python
import threading
import servidor
import json
from servidor.operacoes.somar import Somar
from servidor.operacoes.subtrair import Subtrair
from servidor.operacoes.dividir import Dividir
from servidor.operacoes.multiplicar import Multiplicar
from servidor.operacoes.sqrt import Raizquadrada

class ProcessaCliente(threading.Thread):
    def __init__(self, connection, address):
        super().__init__()
        self.connection = connection
        self.address = address
        self.op_somar = Somar()
        self.op_subtrair = Subtrair()
        self.op_dividir = Dividir()
        self.op_multiplicar = Multiplicar()
        self.op_sqrt = Raizquadrada()

    def receive_int(self, connection, n_bytes: int) -> int:
        data = connection.recv(n_bytes)
        return int.from_bytes(data, byteorder='big', signed=True)

    def send_int(self, connection, value: int, n_bytes: int) -> None:
        connection.send(value.to_bytes(n_bytes, byteorder="big", signed=True))

    def receive_str(self, connection, n_bytes: int) -> str:
        data = connection.recv(n_bytes)
        return data.decode()

    def receive_object(self, connection):
        size = self.receive_int(connection, servidor.INT_SIZE)
        data = connection.recv(size)
        return json.loads(data.decode('utf-8'))
```

processa_cliente.py

servidor > gestor > processa_cliente.py > ProcessaCliente > run

```python
class ProcessaCliente(threading.Thread):
    def run(self):
        while not last_request:
            try:
                request_type = self.receive_str(self.connection, servidor.COMMAND_SIZE)
                if not request_type:
                    break

                if request_type == servidor.OBJ_OP:
                    pedido = self.receive_object(self.connection)
                    operador = pedido["oper"]
                    x = pedido["op1"]
                    y = pedido.get("op2", 0)

                    print(f"[{self.address}] Operação: {x} {operador} {y}")

                    if operador == "+":
                        resultado = self.op_somar.executar(x, y)
                    elif operador == "-":
                        resultado = self.op_subtrair.executar(x, y)
                    elif operador == "*":
                        resultado = self.op_multiplicar.executar(x, y)
                    elif operador == "/":
                        resultado = self.op_dividir.executar(x, y)
                    elif operador == "sqrt":
                        resultado = self.op_sqrt.executar(x)
                    else:
                        resultado = 0

                    self.send_int(self.connection, int(resultado), servidor.INT_SIZE)

                elif request_type == servidor.END_OP:
                    last_request = True
            except Exception as e:
                print(f"[{self.address}] Erro: {e}")
                break

        print(f"[{self.address}] Thread terminada")
        self.connection.close()
```

ii. Coloque este novo ficheiro no lado do servidor



1 (b) Altere o código do ficheiro maquina.py.

i. Introduza esta parte do código no método execute() substituindo o que estava anteriormente:

```
def execute(self):
        print("Waiting for clients on port " + str(servidor.PORT)) while
        True: # Loop infinito para múltiplos clients
            print("On accept...")
            connection, address = self.s.accept()
            print("Client", address, " connected")
            processa = ProcessaCliente(connection, address)
            processa.start() # Arranca thread após ligação
```

ii. Introduza esta parte do código no método execute() substituindo o que estava anteriormente:

```
def execute(self):
        print("Waiting for clients on port " + str(servidor.PORT)) while
        True: # Loop infinito para múltiplos clients
            print("On accept...")
            connection, address = self.s.accept()
            print("Client", address, " connected")
```

```
        processa = ProcessaCliente(connection, address)
        processa.start() # Arranca thread após ligação
```

iii. Introduza esta parte do código no método execute() substituindo o que estava anteriormente:

```
def execute(self):
    print("Waiting for clients on port " + str(servidor.PORT)) while
    True: # Loop infinito para múltiplos clients
        print("On accept...")
        connection, address = self.s.accept()
        print("Client", address, " connected")
        processa = ProcessaCliente(connection, address)
        processa.start() # Arranca thread após ligação
```

iv. Elimine a parte do código que já não é necessária neste ficheiro tendo em conta que agora o processamento das mensagens recebidas do lado do servidor se fazem no novo ficheiro processa_cliente.py.

Resposta:

maquina.py

servidor > gestor > maquina.py > ...

```python
import socket
import servidor
from servidor.gestor.processa_cliente import ProcessaCliente

class Maquina:
    def __init__(self):
        self.s = socket.socket()
        self.s.bind(('', servidor.PORT))

    def execute(self):
        self.s.listen(5)
        print(f"Waiting for clients on port {servidor.PORT}")

        while True:
            print("On accept...")
            connection, address = self.s.accept()
            print(f"Client {address} connected")

            # Start thread for each client (Guide Part A requirement)
            processa = ProcessaCliente(connection, address)
            processa.start()

if __name__ == "__main__":
    maq = Maquina()
    maq.execute()
```

1(c) Corra o servidor e os dois clientes

i. *Corra primeiro o servidor e depois corra dois clientes.*

ii. *Descreva os resultados obtidos do lado dos clientes e também do lado do servidor, com base nas mensagens recebidas.*

Resposta: Ao correr o servidor e dois clientes em simultâneo, o servidor não fica mais bloqueado à espera que um único cliente termine. Devido à estrutura while True e ao uso de processa.start(), o servidor aceita uma ligação, cria uma nova thread ProcessaCliente e volta imediatamente ao estado accept() para ouvir novos clientes. Vê-se mensagens de "Thread iniciada" para cada cliente que se conecta.
Ambos os clientes podem realizar operações de soma ou subtração em paralelo, recebendo respostas independentes do servidor sem terem de esperar pela libertação do recurso por outro utilizador.

iii. *Altere o cliente de forma a ter um ciclo onde mais do que uma operação é pedida. Apresente os resultados obtidos.*

```python
interface.py ×
cliente > Interface > interface.py > ...
class Interface:
    def execute(self):
        while True:
            print("\nEscolha a operação: + - * / sqrt (ou 'sair' para terminar)")
            operacao = input("Operação: ").strip().lower()

            if operacao == "sair":
                self.send_str(cliente.END_OP)
                self.connection.close()
                print("Conexão terminada.")
                break

            try:
                x = float(input("x = "))
                y = 0
                if operacao != "sqrt":
                    y = float(input("y = "))

                self.send_str(cliente.OBJ_OP)
                pedido = {
                    "oper": operacao,
                    "op1": x,
                    "op2": y
                }
                self.send_object(pedido)

                resultado = self.receive_int(cliente.INT_SIZE)
                print(f"Resultado: {resultado}")
            except ValueError:
                print("Erro: Introduza números válidos.")
            except Exception as e:
                print(f"Erro na comunicação: {e}")
                break
```

2. Programar CALCULADORA_6

Pretende-se agora registar os dados de todas as interações que a máquina faz com os clientes. Esses dados irão ser armazenados num dicionário onde se regista para cada interação o tipo de operação bem como os parâmetros e o resultado obtido.

2 (a) Crie a nova pasta dados e dentro dessa pasta introduza o ficheiro dados.py:

```python
import threading
import time

class Dados:
    def _init_(self): self.operacoes
        = {} self.lock =
        threading.Lock()

    def registar_oper(self, oper: str, a: int, b: int, result: int, client:
tuple, timestamp: float = None):
        if timestamp is None:
            timestamp = time.time()
        registo = [a, b, result, client, timestamp]
        with self.lock:
            if oper not in self.operacoes:
                self.operacoes[oper] = []
            self.operacoes[oper].append(registo)
```

dados.py

servidor > dados > dados.py > ...

```python
import threading
import time

class Dados:
    def __init__(self):
        self.operacoes = {}
        self.lock = threading.Lock()

    def registar_oper(self, oper: str, a: int, b: int, result: int, client: tuple, timestamp: float = None):
        if timestamp is None:
            timestamp = time.time()

        registo = [a, b, result, client, timestamp]

        with self.lock:
            if oper not in self.operacoes:
                self.operacoes[oper] = []
            self.operacoes[oper].append(registo)
```

i. Explique que tipos de dados esta classe guarda e descreva a estrutura do dicionário.

Resposta: A classe Dados guarda informações sobre as operações realizadas no sistema. É um dicionário onde a chave é o tipo de operação (ex: 'soma') e o valor é uma lista de registos. Cada item na lista é uma sub-lista contendo: [valor_a, valor_b, resultado, endereço_cliente, timestamp].

ii. Para que serve o lock()? E por que razão é importante utilizá-lo?

Resposta: O lock() serve para garantir a exclusão mútua ao aceder ao dicionário self.operacoes. Em servidores multi-thread, várias threads podem tentar escrever no mesmo dicionário ao mesmo tempo. Sem o lock, podiam ocorrer race conditions, resultando em perda de dados ou corrupção da estrutura do dicionário

2 (b) Altere o ficheiro maquina.py de acordo com este novo ficheiro que guarda estado do sistema.

```python
class Maquina:
    def _init_(self): self.sum
        = Somar() self.dados =
        Dados()
        # ... resto igual (socket)

    def execute(self):
        # ... accept loop
        processo_cliente = ProcessaCliente(connection, address, self.dados)
        processo_cliente.start()
```

servidor > maquina > maquina.py > ...

```python
import socket
import servidor
from servidor.processa_cliente import ProcessaCliente
from servidor.dados.dados import Dados
from servidor.operacoes.somar import Somar

class Maquina:
    def __init__(self):
        self.sum = Somar()
        self.dados = Dados()
        self.s = socket.socket()
        self.s.bind(('', servidor.PORT))

    def execute(self):
        self.s.listen(5)
        print("Waiting for clients on port " + str(servidor.PORT))

        while True:
            print("On accept...")
            connection, address = self.s.accept()
            print("Client", address, "connected")

            processo_cliente = ProcessaCliente(connection, address, self.dados)
            processo_cliente.start()

if __name__ == "__main__":
    maq = Maquina()
    maq.execute()
```

i. Esta alteração faz com que se possa inicializar o objeto self.dados. Esta é uma instância única para todas as thread? Porquê?

Resposta: Sim, o objeto self.dados é uma instância única para todas as threads. Isto acontece porque ele é inicializado na classe Maquina e a mesma referência de memória é passada para cada thread ProcessaCliente que é criada no loop de aceitação

ii. A thread processo_cliente arranca e vai passar os dados. Por que razão o self.dados é enviado como parâmetro?

Resposta: O self.dados é enviado como parâmetro para que a thread ProcessaCliente tenha uma referência ao espaço de memória partilhado. Isto permite que a thread registe as operações efetuadas num local centralizado que persiste mesmo após a thread terminar a sua execução.

2(c) Altere agora o ficheiro processa_cliente.py de forma a que passe a registar os dados das operações efetuadas.

```
class ProcessaCliente(threading.Thread):
    def _init_(self, connection, address, dados):
        super()._init_()
        self.connection = connection
        self.address = address
        self.sum = Somar()
        self.dados = dados


        ...
        if request_type == servidor.ADD_OP:
                x = self.receive_int(servidor.INT_SIZE) y =
                self.receive_int(servidor.INT_SIZE)
                print(self.address,":somar ",x," + ", y)
                result = self.sum.execute(x, y)
                self.send_int(result, servidor.INT_SIZE) #
                Regista soma
                self.dados.registar_oper('soma', x, y, result, self.address) print(self.address,":
                registada uma soma")
```

servidor > processa_cliente.py > ...

```python
import threading
import servidor
import json
from servidor.operacoes.somar import Somar
from servidor.operacoes.subtrair import Subtrair

class ProcessaCliente(threading.Thread):
    def __init__(self, connection, address, dados):
        super().__init__()
        self.connection = connection
        self.address = address
        self.sum = Somar()
        self.dados = dados

    def receive_int(self, n_bytes: int) -> int:
        data = self.connection.recv(n_bytes)
        return int.from_bytes(data, byteorder='big', signed=True)

    def send_int(self, value: int, n_bytes: int) -> None:
        self.connection.send(value.to_bytes(n_bytes, byteorder="big", signed=True))

    def receive_str(self, n_bytes: int) -> str:
        data = self.connection.recv(n_bytes)
        return data.decode()
```

servidor > processa_cliente.py > ...

```python
class ProcessaCliente(threading.Thread):
    def run(self):
        print(self.address, "Thread iniciada")
        last_request = False
        while not last_request:
            request_type = self.receive_str(servidor.COMMAND_SIZE)

            if request_type == servidor.ADD_OP:
                x = self.receive_int(servidor.INT_SIZE)
                y = self.receive_int(servidor.INT_SIZE)
                print(self.address, ":somar ", x, "+ ", y)
                result = self.sum.execute(x, y)
                self.send_int(result, servidor.INT_SIZE)
                # Regista soma
                self.dados.registar_oper('soma', x, y, result, self.address)
                print(self.address, ": registada uma soma")

            elif request_type == servidor.SUB_OP:
                x = self.receive_int(servidor.INT_SIZE)
                y = self.receive_int(servidor.INT_SIZE)
                # Not in guide but logical extension
                sub = Subtrair()
                result = sub.execute(x, y)
                self.send_int(result, servidor.INT_SIZE)
                self.dados.registar_oper('subtrair', x, y, result, self.address)

            elif request_type == servidor.END_OP:
                last_request = True

        print(self.address, "Thread terminada")
        print("Dicionario de dados:", self.dados.operacoes)
        self.connection.close()
```

i. Explique as alterações efetuadas.

Resposta: A classe agora recebe e armazena a referência ao objeto de dados partilhado. Após realizar o cálculo (ex: self.sum.execute), a thread chama explicitamente self.dados.registar_oper para gravar os detalhes da transação antes de enviar o resultado ou continuar o ciclo.

ii. Que tipo de registo é efetuado na classe dados?

Resposta: O registo efetuado é um registo de estado histórico. Ele guarda os números e o resultado, mas também o contexto da operação, quem a pediu (self.address) e quando foi realizada (timestamp).

iii. Coloque o programa a funcionar. E permita que o servidor vá apresentando os resultados guardados nessa estrutura de dados.

iv. Sempre que o cliente termina a operação, mostre o resultado dos dados imprimindo o dicionário que está guardado nos dados.

```
PS C:\Users\Gigabyte\Documents\UAç\2ano_2semestre\Sistemas Distribuídos\LAB's\Calculadora_6>
 py -m servidor.__main__
Waiting for clients on port 35000
On accept...
Client ('127.0.0.1', 64285) connected
('127.0.0.1', 64285) Thread iniciada
On accept...
Histórico: {'subtrair': [[9, 3, 6, ('127.0.0.1', 64285), 1776529646.2781708]]}
('127.0.0.1', 64285) :somar  9 +  3
('127.0.0.1', 64285) : registada uma soma
Histórico: {'subtrair': [[9, 3, 6, ('127.0.0.1', 64285), 1776529646.2781708]], 'soma': [[9,
3, 12, ('127.0.0.1', 64285), 1776529650.3970785]]}
```

```
PS C:\Users\Gigabyte\Documents\UAç\2ano_2semestre\Sistemas Distribuídos\LAB's\Calculadora_6>
 py -m cliente.__main__
Executing Main in cliente
Connected to server at localhost:35000

Select operation: + - (or 'stop' to exit)
Op: -
x = 9
y = 3
Result: 6

Select operation: + - (or 'stop' to exit)
Op: +
x = 9
y = 3
Result: 12

Select operation: + - (or 'stop' to exit)
Op:
```

[Link para o GitHub]

LABORATÓRIO 3 parte A

Sistemas Distribuídos | 2025-2026 | Licenciatura em Informática | FCT UAC

GRUPO:

#1:

#2:

#3:

Instruções de entrega

Indique as alterações realizadas no código que considera mais significativas, apresentando imagens . Não necessita de apresentar o código todo. Deve explicar essas alterações no texto. Identifique cada uma das respostas com a identificação das perguntas, 1a), 1b), etc.

Nota: Deve disponibilizar apontador para o Github onde guarda toda a informação numa pasta LAB1.

LABORATÓRIO 3 parte A

Sistemas Distribuídos | 2025-2026 | Licenciatura em Informática | FCT UAC

GRUPO:

#1:

#2:

#3:

DATA DE ENTREGA:

VALIDAÇÃO DE CONHECIMENTOS

Deve assinalar as respostas verdadeiras justificando

1 Usar as thread

< a completar>

LABORATÓRIO 3 parte A

Sistemas Distribuídos | 2025-2026 | Licenciatura em Informática | FCT UAC

Declaração

Declaro que o trabalho realizado é da nossa autoria. A utilização da IA Generativa está assinalada com a identificação do sistema ou plataforma, a questão ou questões colocadas e, sempre que transcrita informação ela aparece entre-aspas como se tratasse de um autor específico. Mais se declara que se adicionam as referências consultadas.

Ponta Delgada, , Março, 2026